МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Е. Е. Колосов
Группа: М8О-312БВ-24
Преподаватель: Шангин Даниил

Москва, 2026

Условие

Цель работы — разобраться, как задать трёхмерную фигуру, вывести её на экран и управлять её преобразованиями. Для этого нужно написать приложение на C++ с использованием Vulkan, GLFW и Dear ImGui.

Мой вариант — №10, правильный додекаэдр. У него 20 вершин, 30 рёбер и 12 пятиугольных граней. Помимо самой фигуры, в программе сделаны все дополнительные задания:

1. Переключение перспективной и ортографической проекции.

2. Изменение положения, поворота и масштаба по трём осям.

3. Движение по сложной траектории и вращение с запуском, паузой и настройкой скорости и параметров траектории.

4. Изменение цвета фигуры через интерфейс.

5. Вычисление цветов вершин по локальным координатам и умножение на выбранный пользователем цвет.

6. Отрисовка двух объектов с отдельными наборами VkDescriptorSet.

Метод решения

Построение геометрии

За основу я взял стартовый проект vulkan-starter-app преподавателя. В нём уже есть создание окна и подготовка Vulkan. Геометрию фигуры вынес в dodecahedron.hpp. Координаты вершин заданы через золотое сечение:

$$\varphi = \frac{1 + \sqrt{5}}{2}$$

Используются точки вида (±1, ±1, ±1), (0, ±1/φ, ±φ), (±1/φ, ±φ, 0) и (±φ, 0, ±1/φ). Для каждой грани записаны номера пяти точек, которые её образуют.

Видеокарта рисует треугольники, поэтому каждый пятиугольник я разбил на три: (0, 1, 2), (0, 2, 3) и (0, 3, 4). Получилось 36 треугольников и 108 индексов. Для каждой грани нужны свои нормали, поэтому в буфере хранятся 60 записей Vertex. Самих геометрических вершин при этом по-прежнему 20.

Преобразования и проекция

Положение, поворот и масштаб меняются в интерфейсе. Функция modelMatrix в application.cpp собирает из них матрицу модели. Сначала фигура растягивается, затем поворачивается вокруг осей X, Y и Z, после чего переносится в нужную точку:

$$M = T \cdot R_z \cdot R_y \cdot R_x \cdot S$$

Вершинный шейдер применяет к каждой точке три матрицы: M задаёт преобразования фигуры, V — положение относительно камеры, P — проекцию. В результате получаются координаты в пространстве отсечения:

$$p_{clip} = P \cdot V \cdot M \cdot p_{local}$$

Матрица V сдвигает сцену на 6 единиц от камеры. В sceneMatrix я сделал выбор между двумя матрицами P. Перспектива даёт привычный эффект: при удалении фигура становится меньше. В ортографической проекции её размер от расстояния не зависит.

Для перспективы угол обзора равен 45°. Ортографическая область подобрана так, чтобы на расстоянии 6 размеры были близкими. Ближняя плоскость отсечения равна 0,1, дальняя — 100. Также учтены особенности Vulkan: глубина от 0 до 1 и направление оси Y. Соотношение ширины и высоты окна учитывается при построении обеих проекций.

Анимация

Чтобы движение не зависело от FPS, в update считается время между кадрами. Пока нажато Play, время анимации t увеличивается с учётом Speed. Для движения по трём осям используются синусы с разными частотами:

$$\Delta x = r \cdot \sin(t)$$
$$\Delta y = h \cdot \sin(2t)$$
$$\Delta z = \frac{r}{2} \cdot \sin(3t)$$

Здесь r меняется ползунком Radius, а h — ползунком Height. Получается объёмная петляющая траектория. Смещение добавляется к позиции, заданной вручную. Одновременно фигура вращается вокруг Y со скоростью Spin. На паузе время останавливается, а Reset animation возвращает фигуру к ручной позиции и повороту. У двух объектов анимация работает независимо.

Цвет и освещение

Цвета вершин вычисляются в dodecahedron.vert по исходным координатам точек. Отрицательные координаты нельзя напрямую использовать как цвет, поэтому диапазон от −φ до φ переводится в диапазон от 0 до 1:

$$C_{\text{vertex}} = \text{clamp}(\frac{p_{\text{local}}}{2\varphi} + \frac{1}{2}, 0, 1)$$

X отвечает за красный канал, Y — за зелёный, Z — за синий. Между вершинами цвет плавно смешивается. При движении цвета остаются на своих местах, поскольку берутся локальные координаты. Выбранный Figure color умножается на цвет вершины. Если отключить Vertex colors, фигура становится однотонной.

Я также добавил простой направленный свет, чтобы грани лучше различались. При повороте и растяжении нормали пересчитываются обратной транспонированной матрицей линейной части M. Это нужно, чтобы освещение не искажалось при разном масштабе по осям. Фрагментный шейдер получает цвет и умножает его на яркость грани.

Объекты Vulkan

Вершины и индексы записываются в два VkBuffer, память для них выделяется через VMA. В VkPipeline задаются шейдеры, формат вершин и способ рисования треугольников. Проверка глубины нужна, чтобы ближние грани закрывали дальние. При изменении размера окна обновляется область рисования.

Для шестого задания я использовал два додекаэдра с общей геометрией. Функция createDescriptors создаёт VkDescriptorSetLayout и пул VkDescriptorPool на два набора. У каждого объекта свой uniform-буфер и свой VkDescriptorSet. В буфер записываются матрицы и цвет, а дескриптор указывает шейдеру, откуда их читать.

В render сначала обновляются данные обоих объектов. Затем для каждой фигуры вызывается vkCmdBindDescriptorSets с её набором и vkCmdDrawIndexed. Поэтому настройки одной фигуры не влияют на другую. Перед обновлением буферов программа ждёт завершения прошлого кадра, а при закрытии освобождает созданные ресурсы в shutdown. Основная логика находится в application.cpp, а запуск Vulkan и вывод кадров — в graphics_internal.cpp стартового проекта.

Результаты

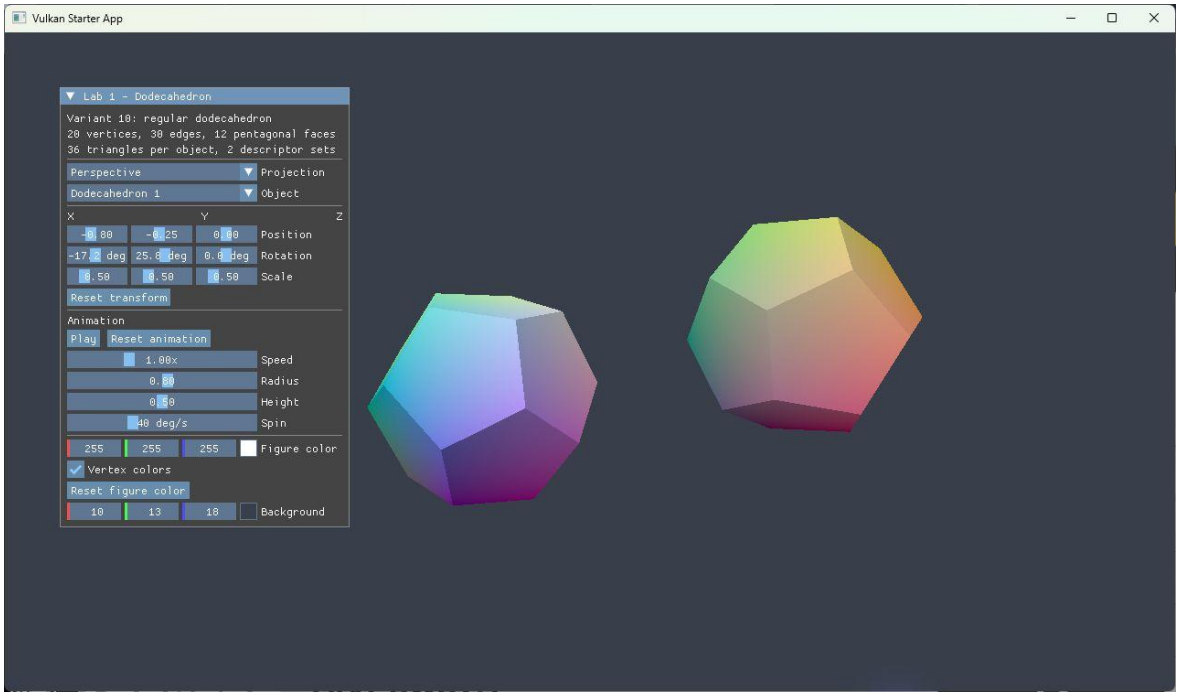


Рисунок 1. Два додекаэдра в перспективной проекции. Положение, поворот, масштаб и цвет настраиваются отдельно для каждой фигуры.

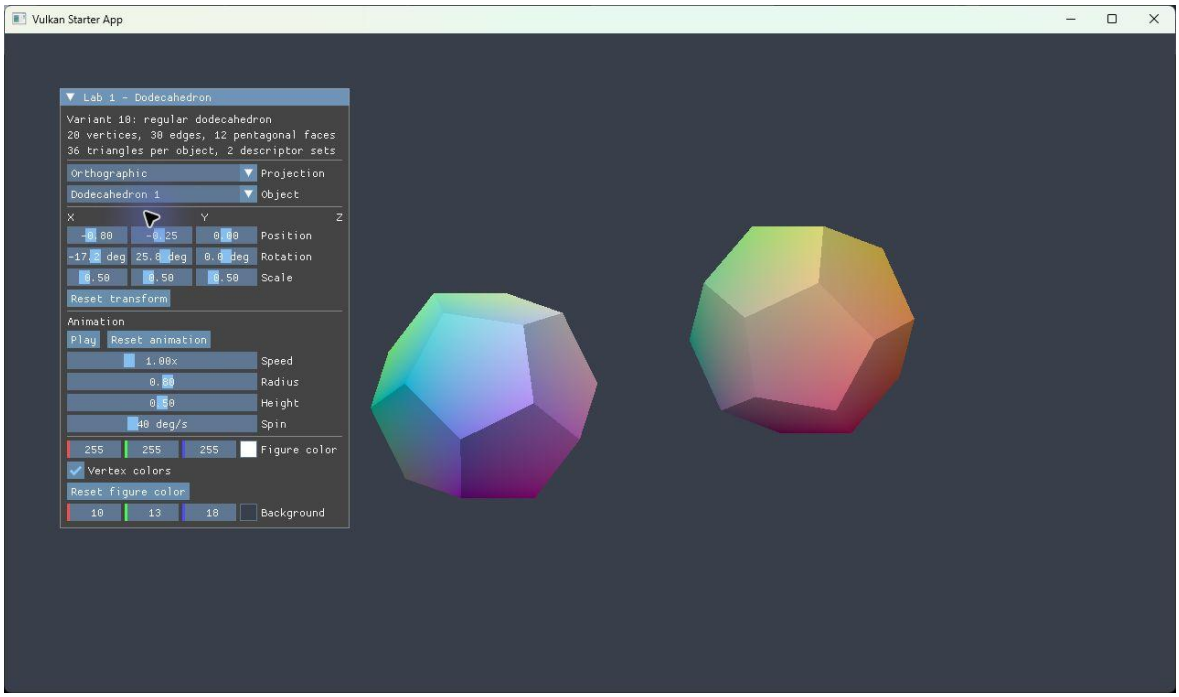


Рисунок 2. Та же сцена после переключения на ортографическую проекцию. Настройки объектов остались прежними.

Управление объектами

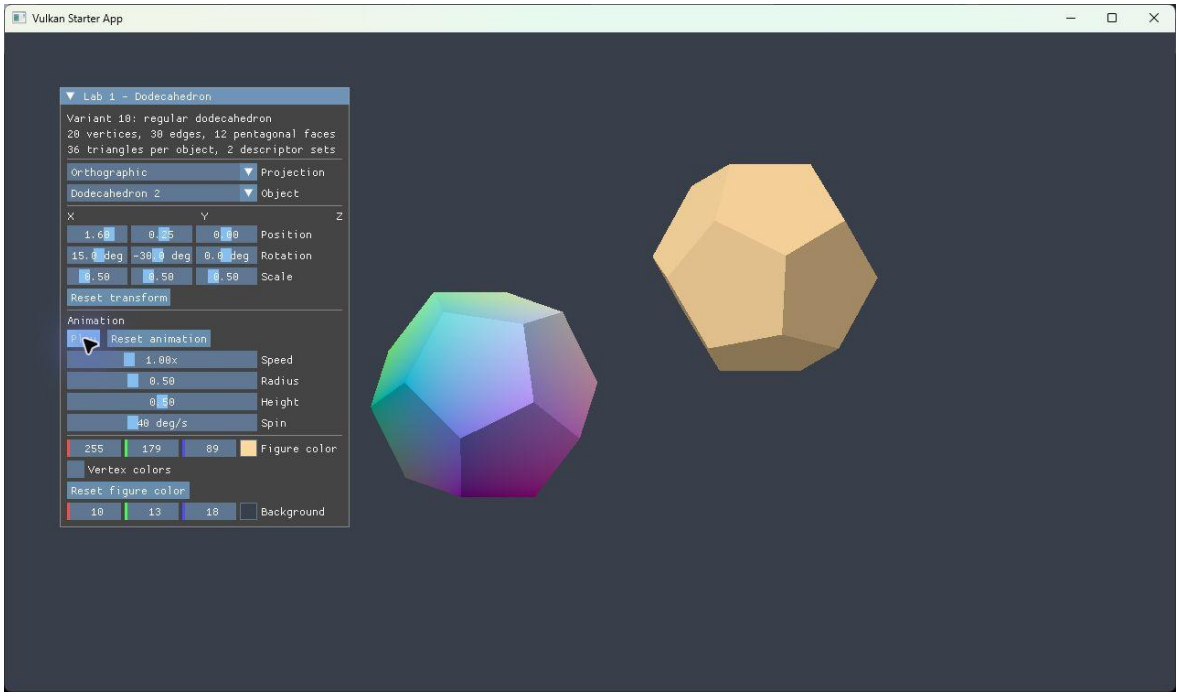


Рисунок 3. Второй додекаэдр остановлен на паузе после анимации. Для него выбран однотонный цвет, а первый объект остался разноцветным.

Программа собрана в CLion с компилятором MSVC. Проверены обе проекции, изменение параметров каждой фигуры, анимация, пауза и сброс. Также проверены изменение размера окна и закрытие программы. При этих проверках ошибок валидации Vulkan не было.

Выводы

В результате получилась программа, которая рисует правильный додекаэдр и позволяет управлять им в реальном времени. Выполнен вариант 10 и все шесть дополнительных заданий: переключение проекций, преобразования, анимация, выбор цвета, цвета вершин и два объекта с разными наборами дескрипторов.

В этой работе я использовал матрицы для преобразования фигуры и вывода её на экран, разбил пятиугольные грани на треугольники и связал параметры интерфейса с отрисовкой. На примере двух объектов видно, что одну геометрию можно рисовать несколько раз с разными матрицами и цветами, если перед каждым рисованием подключать нужный VkDescriptorSet.